

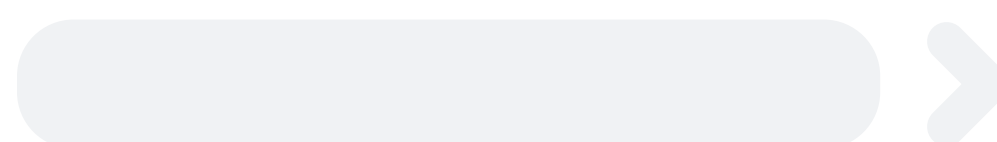
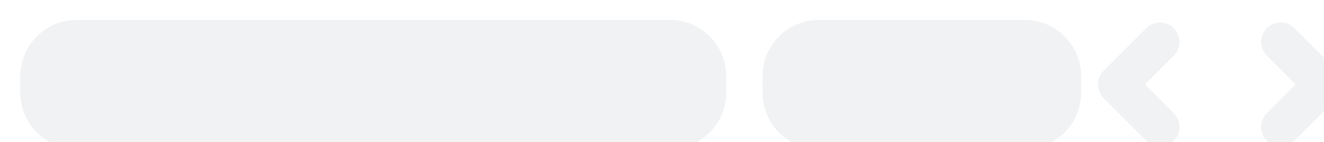
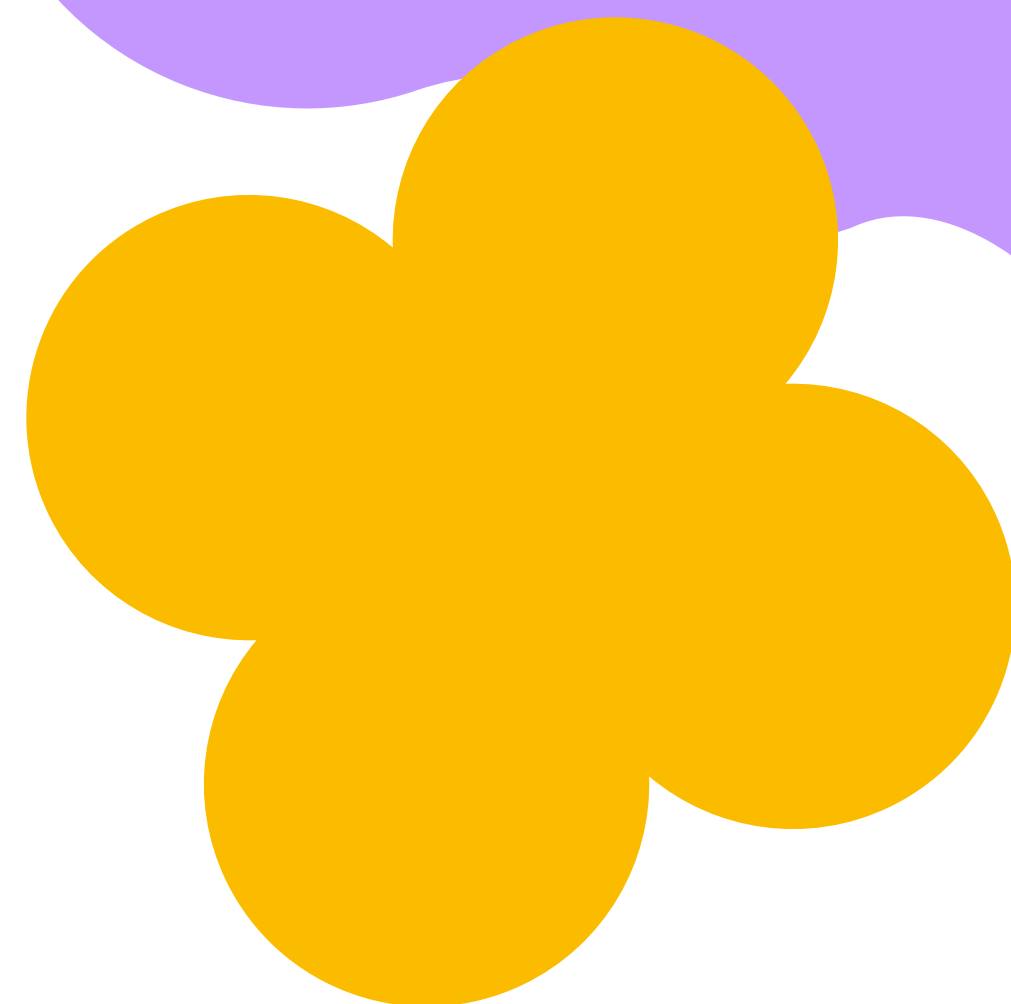
The solution:

Session state

Reference: [ADK documentation - session state](#)

Conceptually, `session.state` is a collection (dictionary or map) holding key-value pairs. It's designed for information the agent needs to recall or track to make the current conversation effective:

- **Personalize interaction:** Remember user preferences mentioned earlier (e.g., `'user_preference_theme': 'dark'`).
- **Track task progress:** Keep tabs on steps in a multi-turn process (e.g., `'booking_step': 'confirm_payment'`).
- **Accumulate information:** Build lists or summaries (e.g., `'shopping_cart_items': ['book', 'pen']`).
- **Make informed decisions:** Store flags or values influencing the next response (e.g., `'user_is_authenticated': True`).



The solution: Session state

What is session state?

Session state is a Python dictionary accessible through the `session.state` attribute. Think of it as variables your agent stores and your code can check:

```
Python
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

# Create agent with output_key to
# save response
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="Extract the user's name. Return ONLY the name.",
    output_key="user_name" # Saves response to state["user_name"]
)

# Setup session and runner
session_service = InMemorySessionService()
session = session_service.create_session(app_name="my_app", user_id="user1")
runner = Runner(agent=agent, app_name="my_app", session_service=session_service)

# Run agent
result = runner.run(user_id="user1", session_id=session.id, new_message="My name is
Alex")

# Access state from outside the agent
print(session.state) # {'user_name': 'Alex'}
print(session.state.get("user_name")) # 'Alex'

# Now your code can make decisions:
if session.state.get("user_name"):
    print("✅ User has provided their name!")
```

What state enables

- ✅ **Programmatic access** - Your code can read exact values with `session.state.get("key")`
- ✅ **Automatic saving** - Use `output_key` to save agent responses without manual code
- ✅ **Decision making** - Check state values to make programmatic decisions
- ✅ **Data persistence** - Values remain accessible across conversation turns

Core concepts

1. State vs. conversation history

Both are available to your agents, but they serve different purposes:

Feature	Conversation history	Session state
What it is	Text messages (user + agent)	Dictionary of key-value pairs
LLM can read	✔ Yes (automatic)	✔ Yes (via <code>{var}</code> templating, Part 2)
Code can check	✘ No	✔ Yes (<code>session.state.get("key")</code>)
Use for	Context for LLM	Exact values for your code
Access	Automatic (sent to model)	Programmatic (<code>session.state</code>)

Visual comparison

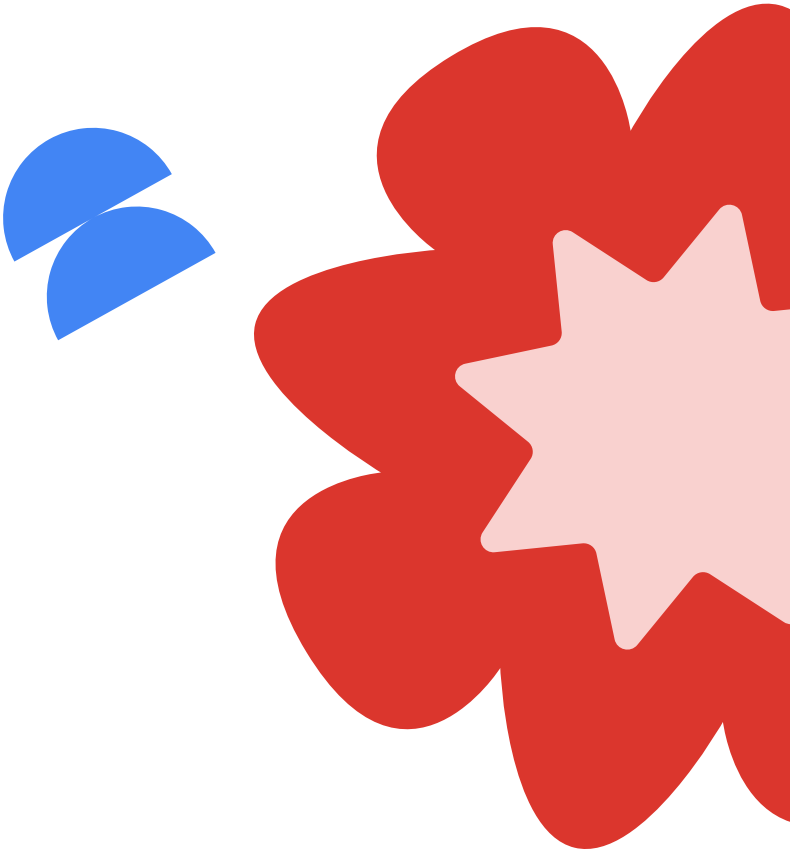
```
None
graph TB
    subgraph "Conversation History"
        CH1[Text messages<br/>User and Agent]
        CH2[LLM reads ✔]
        CH3[Your code CANNOT access ✘]
    end

    subgraph "Session State"
        SS1[Dictionary<br/>key-value pairs]
        SS2[LLM can read ✔]
        SS3[Your code CAN access ✔]
    end

    style CH2 fill:#e1f5ff
    style CH3 fill:#ffeeee
    style SS2 fill:#e1f5ff
    style SS3 fill:#eeffee
```

When to use state

- ✔ When your code needs to check exact values
(`if state.get("name"):`)
- ✔ When making routing decisions programmatically
- ✔ When tracking conversation progress or user information
- ✘ DON'T use for: Only providing context to LLM (conversation history already does that)



2. Accessing state with `session.state`

Reference: [ADK documentation - state access](#)

After running your agent, you access state through the `session.state` dictionary:

```
Python
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Create session
session_service = InMemorySessionService()
session = session_service.create_session(app_name="app", user_id="user1")

# Run agent
runner = Runner(agent=my_agent, app_name="app",
session_service=session_service)
result = runner.run(
    user_id="user1",
    session_id=session.id,
    new_message=Content(parts=[Part(text="Hello")])
)

# Access state AFTER agent runs
print(session.state) # Full state dictionary
name = session.state.get("user_name", "Guest") # Safe read with default
count = session.state.get("count", 0) # Returns 0 if key doesn't exist
```

Key points

- 1 State is accessed as a dictionary:
`session.state`
- 2 Use `.get(key, default)` for safe reading
(returns default if key doesn't exist)
- 3 State persists across all turns in the
same session
- 4 Your code can check state values to make
programmatic decisions



3. Saving responses with `output_key`

Reference: [ADK documentation - output key](#)

From ADK documentation

“`output_key` (optional): Provide a string key. If set, the text content of the agent's final response will be automatically saved to the session's state dictionary under this key. This is useful for passing results between agents or steps in a workflow.”

The simplest way to save an agent's response to state is using `output_key`:

```
Python
from google.adk.agents import LlmAgent

agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="Extract the main topic. Return ONLY the topic.",
    output_key="topic" # Automatically saves response to state["topic"]
)

# After agent runs, state["topic"] contains the response text
# No manual code needed!
```

How it works

- 1 Agent generates response (e.g., “quantum computing”)
- 2 Framework automatically saves to `state["topic"] = "quantum computing"`
- 3 You can access it with `session.state.get("topic")`

When to use

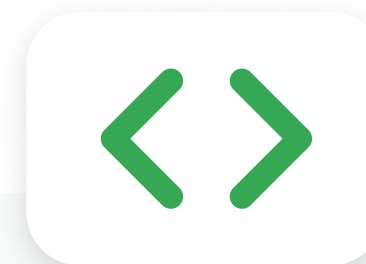
- ✓ When you want to save the agent's response for later use
- ✓ When you need programmatic access to what the agent said
- ✓ When building applications that need to track extracted information
- ✗ DON'T use for: Responses you don't need to reference later

Hands-on example

Building a name extraction agent

What you'll build: A simple agent that extracts a user's name and saves it to state, demonstrating how state enables programmatic access.

Step 1: Create the project



```
Shell
adk create name_extractor
cd name_extractor
```

Step 2: Write the agent

Replace the contents of `agent.py` with:

```
Python
"""
Name Extractor - Demonstrates Session State Basics
Shows how to use output_key to save data and access it via session.state.

Reference: https://google.github.io/adk-docs/sessions/state.md
"""

from google.adk.agents import LlmAgent

# Single agent that extracts and saves name
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='name_extractor',
    instruction="Extract the person's name from the message. Return ONLY the name, nothing else.",
    output_key="user_name" # Saves response to state["user_name"]
)
```

Code walkthrough

Lines 10-16: Name extractor agent

- Uses `output_key="user_name"` to automatically save the response
- After this agent runs, `state["user_name"]` contains the extracted name
- No manual state management code needed!

Step 3: Test with runner

Instead of `adk web`, let's use the `Runner` directly to see state access:

Create a file called `test_state.py`:

Python

"""

Test script to see state access
directly.

Run with: `python test_state.py`

"""

```
from agent import root_agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part
```

Setup session and runner

```
session_service = InMemorySessionService()
session = session_service.create_session(
    app_name="name_extractor_app",
    user_id="test_user",
    session_id="test_session"
)
```

```
runner = Runner(
    agent=root_agent,
    app_name="name_extractor_app",
    session_service=session_service
)
```

Test: Extract name

```
user_message = Content(parts=[Part(text="Hi, my name is Alex Johnson")])
```

```
print("=== Running agent ===")
result = runner.run(
    user_id="test_user",
    session_id="test_session",
    new_message=user_message
)
```

Show final response

```
for event in result:
    if event.is_final_response():
        print(f"\nAgent response: {event.content.parts[0].text}")
```

Access state programmatically

```
print(f"\n=== State after execution ===")
print(f"Full state: {session.state}")
print(f"Extracted name: {session.state.get('user_name')}")
```

Your code can now make decisions based on state

```
if session.state.get("user_name"):
    print("✅ Name was successfully extracted and stored!")
else:
    print("❌ Name extraction failed")
```

Test accessing in subsequent turns

```
print("\n=== Simulating second turn ===")
result2 = runner.run(
    user_id="test_user",
    session_id="test_session",
    new_message=Content(parts=[Part(text="What's my name?")])
)
```

```
for event in result2:
    if event.is_final_response():
        print(f"Agent response: {event.content.parts[0].text}")
```

```
print(f"\nState still contains: {session.state.get('user_name')}")
print("✅ State persists across turns!")
```


Run it:

```
Shell  
python test_state.py
```

Expected output:

```
None  
=== Running agent ===  
  
Agent response: Alex Johnson  
  
=== State after execution ===  
Full state: {'user_name': 'Alex  
Johnson'}  
Extracted name: Alex Johnson  
✔ Name was successfully extracted and  
stored!  
  
=== Simulating second turn ===  
Agent response: Your name is Alex  
Johnson  
  
State still contains: Alex Johnson  
✔ State persists across turns!
```

What to notice

- ✔ `output_key="user_name"` automatically saved "Alex Johnson" to state
- ✔ Your code can access `session.state.get("user_name")` programmatically
- ✔ You can make decisions: `if session.state.get("user_name"):`
- ✔ State persists across multiple turns in the same session

Key takeaways

Session state fundamentals:

- **Session state** is a dictionary where agents store data that your code can programmatically access
- **Conversation history** provides context to the LLM but cannot be checked programmatically by your code
- Both are available and complement each other (history for LLM context, state for programmatic control)

How to use state:

- Access state with `session.state` after running agents via Runner
- Use `.get(key, default)` to safely read state values
- Use `output_key="key"` to automatically save agent responses to state
- State persists across all turns in the same session

When to use state

- ✔ When your code needs to check exact values programmatically
- ✔ When making routing decisions based on conversation data
- ✔ When tracking conversation progress or user information
- ✔ When storing extracted information for application logic

Common pattern:

Python

```
# 1. Agent saves to state with  
output_key  
agent = LlmAgent(output_key="result")  
  
# 2. Run agent  
runner.run(...)  
  
# 3. Access state programmatically  
if session.state.get("result"):  
    # Make decisions based on state  
    pass
```

Next part

You'll learn how to use `{var}` templating to inject state values into instructions, enabling dynamic agent behavior.

